MyInfArith:Infinite Precision Calculator

Aryan Agarwal
Roll Number: ES23BTECH11008
GitHub : AryanAg2701

May 2, 2025

1 Introduction

In this project, I built a Java tool called `MyInfArith` that can do math on very big numbers bigger than standard Java types allow. It can add, subtract, multiply, and divide integers and floating-point numbers of any size. I use strings to store the numbers inside the program.

2 Project Layout

Here are the main files in the project:

- `MyInfArith.java`: The main program. It reads your inputs and runs the right calculations.
- `AInteger.java`: A class for doing math on big integers.
- `AFloat.java`: A class for doing math on big decimals.
- `build.xml`: An Ant script that helps you compile, package, and run the code.
- `run_build.py`: A small Python script that calls Ant commands for you.
- `DockerFile`: A small DockerFile which contains list of instructiond to use a Docker image.

3 How It Works

Both `AInteger` and `AFloat` store numbers in two parts:

- `String digits`: The number's digits, without the sign or decimal point.
- `boolean negative`: A flag saying whether the number is negative.

When you call methods like `add`, `sub`, `mul`, or `div`, they handle the sign first, then call helper functions that do the actual digit-by-digit math on the strings.

4 Building and Running

4.1 Using Ant

I use Apache Ant to automate everything. The `build.xml` file has these targets:

- `clean`: Deletes previous build files.

- `compile`: Compiles all the Java files.
- `jar`: Packages the compiled classes into `dist/MyInfArith.jar`.
- `run`: Runs the program with your inputs.

To build and run in one step, just type:

```
ant run -Dargs="int add 123 456"
```

4.2 Using the Python Script

If you prefer Python, you can also do:

```
python3 run_build.py --args "float mul 3.14 2.5"
```

This runs clean, compile, jar, and run for you.

5 Using the Program

First, compile the Java source files directly:

```
javac aribtraryarithmetic/*.java
javac MyInfArith/*.java
```

Then run the main class from the build directory without a JAR:

```
java MyInfArith <type> <operation> <num1> <num2>
```

Where:

- **type**: `int` or `float`
- **operation**: `add`, `sub`, `mul`, or `div`
- **num1**, **num2**: Your numbers as strings (e.g., "123456789012345678 90").

6 Special Cases

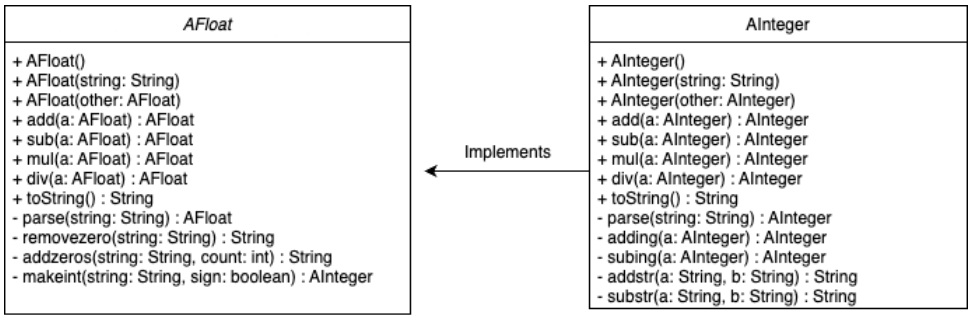
6.1 Integers

If you divide and there's a remainder, I just drop the decimal part (like integer division).

6.2 Floats

If you type numbers like ".89" or "83.", I convert them to "0.89" or "83.0" automatically.

7 Internal Classes



This section explains the two core classes that power all computations: `AInteger` for integer math and `AFloat` for decimal math. Each class stores numbers as strings, handling the sign separately to simplify arithmetic logic.

7.1 AInteger

The `AInteger` class is responsible for handling integer values of arbitrary length. Internally, it stores all digits as a string and the sign as a boolean. This design allows precise control over every digit and mimics manual arithmetic operations at the string level.

Fields:

- `String digits`: Contains the digits of the number as a plain string, without signs or formatting.
- `boolean negative`: True if the number is negative, false otherwise.

Constructors:

- `AInteger()`: Initializes the number to 0 with a positive sign.
- `AInteger(String s)`: Parses a string. If it starts with '-', sets negative = true. It also trims any leading zeros.
- `AInteger(AInteger other)`: Creates a deep copy of another `AInteger`.

Key Methods:

- `add(AInteger other)`:
  - If the signs differ, converts the operation into subtraction.
  - If signs match, uses `addStr(String a, String b)`:
    - Pads the shorter string with zeros to match length.
    - Adds digits from right to left, storing carry.
    - Reverses and trims the result.
- `sub(AInteger other)`:
  - If signs differ, delegates to `add`.
  - If signs match, calls `subStr(String a, String b)`:
    - Compares magnitudes to decide which number to subtract from which.
    - Performs digit-by-digit subtraction with borrow if needed.
- `mul(AInteger other)`:

  - Uses `mulStrByDigit(String a, char d)` to multiply the number by each digit of the other.
  - Shifts the intermediate result based on position.
  - Accumulates with `addStr`.
  - Sign is negative if exactly one input is negative.

- `div(AInteger other)`:
  - Uses `divStr(String a, String b)` to perform long division.
  - Builds the quotient digit by digit.
  - Discards remainder to stay consistent with integer division behavior.

- `static parse(String s)`:
  - Cleans and validates the input string.
  - Returns a new `AInteger` instance with the parsed digits and sign.

- `toString()`:
  - Converts the stored fields back to a user-readable format.
  - Prepends '-' if negative.

7.2 AFloat

The `AFloat` class is designed for handling decimal numbers with arbitrary precision. It extends `AInteger`'s logic by introducing a decimal position field, allowing accurate arithmetic operations even on large or very small float values.

Fields:

- `String digits`: Stores all digits as a string, removing the decimal point for simplified operations.
- `boolean negative`: Indicates the sign of the number.
- `int dec`: Tells how many digits are present after the decimal point.

Constructors:

- `AFloat()`: Initializes the value to zero with dec = 0.
- `AFloat(String s)`:
  - Parses strings like "-12.34", splitting at the decimal point.
  - Combines digits into a single string (e.g., "12.34" becomes "1234" with dec 2).
  - Handles corner cases like ".75" or "83." by prepending or appending zeros.
- `AFloat(AFloat other)`: Creates a deep copy, copying digits, sign, and dec.

Key Methods:

- `add(AFloat other)` and `sub(AFloat other)`:
  - Normalize both numbers to the same dec by adding trailing zeros to the smaller one.
  - Convert both to their integer form and apply `AInteger`'s add/sub.
  - The result's dec remains consistent with the inputs.

- `mul(AFloat other)`:
  - Multiplies integer forms using `AInteger`'s multiplication logic.
  - Sets result's dec as the sum of both operand scales.
- `div(AFloat other)`:
  - Scales both operands to eliminate decimal points.
  - Multiplies numerator by a power of 10 (e.g., 10^1000) for precision.
  - Uses `AInteger.div` to get result and sets dec to fixed precision (e.g., 1000).
- `static parse(String s)`:
  - Validates that input is a properly formatted float.
  - Handles inputs like ".007" by adjusting dec and digit string.
- `toString()`:
  - Converts back to readable format by placing the decimal at the correct position.
  - Adds leading/trailing zeros where necessary and adds "-" if the number is negative.

8 Challenges and Edge Cases

In developing `AFloat` and `AInteger`, several issues required special attention to prevent bugs and unexpected behavior. Below are key challenges and the code strategies used to address them.

1. Handling Leading and Trailing Zeros

When parsing input strings or doing arithmetic, it's important to remove unnecessary zeros, such as "000123" or "123.45000". This avoids incorrect magnitude or floating point misalignment.

2. Parsing Floats with Missing Whole or Decimal Parts

Users might input floats as ".89", "83.", or "000.0050". These cases are handled in the constructor of `AFloat`:

3. Avoiding String Index Errors in Division

When performing string division, it's easy to get `StringIndexOutOfBoundsException` if you don't check lengths. The `divStr` method ensures we only read valid positions:

4. Overriding `toString()` for Clean Output

For both `AInteger` and `AFloat`, overriding `toString()` ensures formatted, human-readable output: `AInteger`:
`AFloat`:

5. Ensures correct digit alignment

By padding the shorter number with leading zeros so both strings are of equal length. This guarantees that digits are added in the correct column positions.

9 Limitations

I can handle very large numbers, but limited by available memory and Java's `String` size. Floats are precise up to 1000 decimal places which can be changed according to choice. If you give inputs in wrong format answer moght be wrong.

10 Testing

I tested with random inputs from online generators and compared results with standard calculators. I tried big numbers, zeros in weird spots (like "000123"), and different decimal formats.

11 Version Control

I used Git for version control. Below is a placeholder for the commit history diagram:

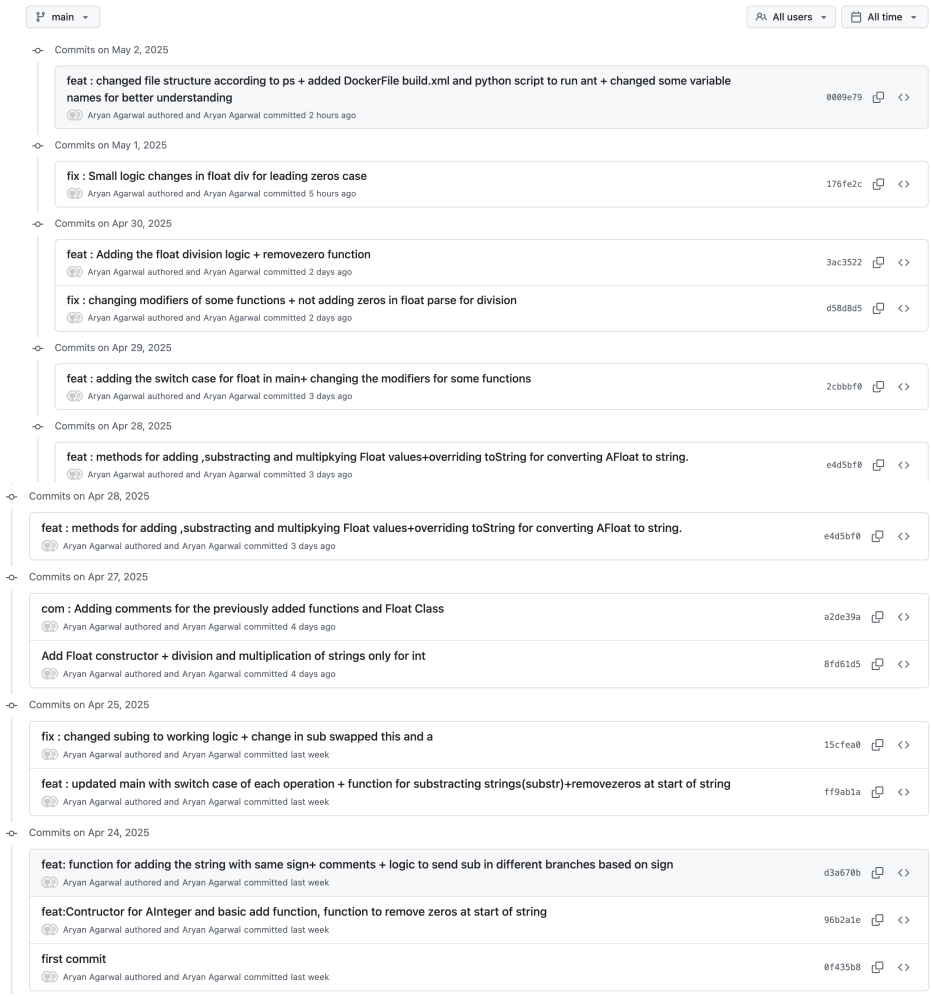


Figure 1: Git Commit History

12 What I Learned

- How to implement big-number math from scratch in Java.

- A lot about classes , methods, contructots etc in Java.
- How to cleanly separate sign logic from digit math.
- How to use Ant and a Python helper script for builds.
- How to use Docker and Git